

Serveur Manager V1 – Gestion centralisée de serveurs de jeux avec Docker & Python

Projet collaboratif – Administration système, Dockerisation & interface web

Contexte du projet

Après plusieurs années à héberger manuellement des serveurs de jeux sur une machine Linux, le processus était devenu :

- long,
- répétitif,
- difficile à maintenir,
- et peu pratique lorsque plusieurs serveurs devaient tourner en même temps.

Avec un ami, nous avons décidé de créer **Serveur Manager V1**, une application permettant de **centraliser la gestion de tous nos serveurs de jeux**, d'automatiser leur lancement, et de simplifier leur maintenance.

Ce projet combine :

- **Python** pour la logique,
- **Docker** pour encapsuler chaque serveur,
- et une **interface web maison** pour gérer le tout.

C'est l'un de mes projets les plus avancés techniquement.

Objectifs du projet

- Automatiser le lancement et l'arrêt des serveurs de jeux.

- Dockeriser chaque jeu pour éviter les conflits de versions(Java pour Minecraft).
- Créer une interface web simple montrant l'état des serveurs.
- Pouvoir gérer plusieurs jeux simultanément, sans surcharge du système.
- Simplifier toutes les opérations :
 - redémarrage,
 - mise à jour,
 - modification de configuration.

L'objectif final : **rendre l'hébergement totalement flexible et professionnel.**

Fonctionnalités principales

✓ 1. Dockerisation des serveurs

Chaque jeu était placé dans un **conteneur Docker** séparé.
Cela permettait de :

- isoler les fichiers,
- éviter les conflits de dépendances,
- lancer/arrêter facilement les serveurs,
- limiter la consommation mémoire.

C'était particulièrement utile pour :

- Minecraft,
- ARK,
- GMod,
- Unturned,
- et d'autres jeux que nous ajoutons au fur et à mesure.

✓ 2. Application Python (backend)

La logique principale du Serveur Manager était écrite en **Python** :

- scripts pour lancer ou arrêter les conteneurs,
- récupération des logs,
- surveillance des performances,
- communication avec l'interface web.

Python permettait de piloter Docker via des commandes automatisées.

✓ 3. Interface web maison

Nous avons développé une **interface web simple**, permettant de :

- voir quels serveurs sont en ligne,
- démarrer ou arrêter chaque serveur,
- afficher des informations comme l'utilisation CPU/RAM,
- ajouter ou retirer des serveurs à la liste.

Cette interface transformait complètement l'expérience :

👉 plus besoin de lancer des commandes Linux à la main.

✓ 4. Gestion multi-serveurs

L'application supportait plusieurs serveurs en parallèle, par exemple :

- un serveur Minecraft,
- un serveur ARK,
- un serveur BeamMP,
- un serveur Unturned,

tous lancés dans leurs conteneurs respectifs.

Cela a rendu la machine beaucoup plus polyvalente.



Ce que ce projet m'a appris

Ce projet est clairement l'un de ceux qui m'ont fait le plus progresser techniquement. J'y ai appris :



Administration système

- gestion avancée d'un OS Linux,
- optimisation d'un serveur multi-processus.



Docker

- création et gestion de conteneurs,
- dockerisation de services complexes,
- volumes, images, container networking.



Python

- automatisation,
- interaction avec Docker via scripts,
- organisation d'un petit backend.



Développement web

- interface de gestion,
- gestion des états des serveurs,
- interaction frontend ↔ backend.



Architecture & organisation

- mise en place d'un vrai système unifié,
 - réflexion sur l'expérience utilisateur (simplicité, accessibilité),
 - industrialisation de tâches auparavant manuelles.
-



Conclusion

Serveur Manager V1 est né d'un objectif simple :

👉 éviter la gestion manuelle constante des serveurs de jeux.

Mais au final, le projet est devenu une **solution complète**, mêlant :

- Docker,
- Python,
- Web,
- administration système,
- automatisation.

Ce projet m'a permis de développer des compétences réelles de **DevOps**, d'administration système et d'organisation logicielle, tout en répondant à un besoin concret de mes amis et moi.

